



Recherche d'Information

Représentation de documents et

Indexation

*Dr Kiswendsida Kisito Kaboré ,
Enseignant Chercheur Au
Département d'informatique
à l'UFR-SEA / UJKZ*

Mai 2022

<http://kisitokab.xyz>



Contenu-sujets du Cours



- Introduction à la RI:
 - notions fondamentales et les concepts
 - problématique
 - historique
- **Représentation de documents et Indexation**
- Modèles de RI
- Traitement des requêtes et Évaluation de la RI
- Moteurs de recherche sur le Web

Objectif

À la fin de ce chapitre, vous devriez être capable de:

- Maîtriser les techniques d'indexation.
- Maîtriser les mécanismes de représentation documentaire.



Chapitre 2 : Représentation de documents et Indexation

- **Introduction**
- Nous nous intéressons
 - aux mécanismes amenant à la représentation documentaire et
 - à l'indexation et qui constituent les briques de base de presque tous les algorithmes classiques en accès à l'information.

Chapitre 2 : Représentation de documents et Indexation

- **Introduction**
- Ces mécanismes comprennent un ensemble de prétraitements permettant :
 - la constitution du vocabulaire à partir d'une collection de documents,
 - la représentation des documents dans l'espace des termes,
 - la construction de l'index inversé des termes ainsi que
 - la compression du vocabulaire et de l'index qui favorise l'accélération des traitements et l'accès aux données.

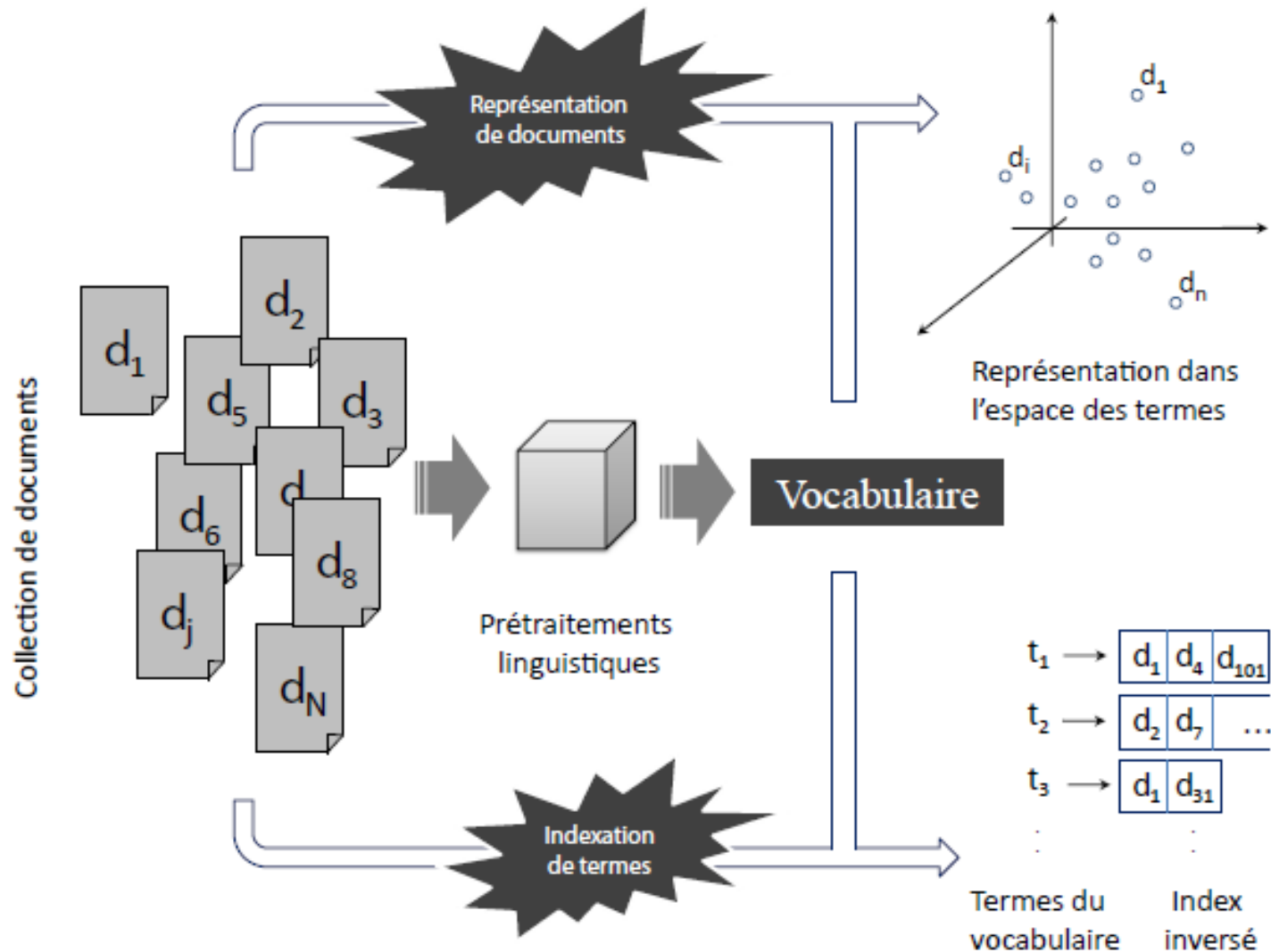


Fig. 3 – Constitution du vocabulaire, index inversé des termes et représentation des documents dans l'espace des termes pour une collection de documents donnée.

2.1) Prétraitements linguistiques

2.1) *Prétraitements linguistiques*

- Introduction
- L'objectif de cette partie est de présenter l'ensemble des prétraitements linguistiques conduisant à la constitution de l'index inversé et de la représentation documentaire à partir d'une collection de documents donnée.

2.1) Prétraitements linguistiques

- Introduction
- Ces prétraitements comprennent
 - la *segmentation* des séquences de caractères présents dans les documents en mots distincts,
 - la *normalisation* des mots, opération qui consiste à fournir une *forme canonique* pour chaque mot,
 - et le *filtrage* qui vise à supprimer les mots les plus fréquents.

2.1.1) La Segmentation

- La segmentation (en anglais *tokenisation*) consiste à séparer une suite de caractères en éléments sémantiques, ou **mots**.
- **Un type** de mot est la classe de tous les mots ayant la même séquence de caractères et
- **un terme** est un type de mots que l'on garde pour former le vocabulaire.

2.1.1) La Segmentation

- Soit l'exemple ci-dessous :

*Combien de mots
avons-nous ?*

***Au sens étymologique, l'information est ce
qui donne une forme à l'esprit***

2.1.1) La Segmentation

Au sens étymologique, l'information est ce qui donne une forme à l'esprit

- nous avons 14 mots : *Au, sens, étymologique, l', information, est, ce, qui, donne, une, forme, à, l', esprit*
- mais seulement 13 types, puisqu'il y a deux instances de {*l'*}.
- Après filtrage par un **antidictionnaire**, il ne restera plus que 6 termes

{*sens, étymologique, information, donne, forme, esprit*}
dans le vocabulaire.

2.1.1) *La Segmentation*

- Cette étape de segmentation est une étape difficile et cruciale puisqu'une mauvaise segmentation a un impact négatif direct sur le résultat de la recherche.
- En effet, si dans une collection certains termes ne sont pas indexés, les requêtes composées de ces termes ne pourront jamais être appariées avec les documents de la collection contenant ces termes.

2.1.1) La Segmentation

- Une bonne segmentation dépend de la prise en compte des spécificités de la langue des textes traités.
- Chaque langue a sa spécificité propre et une simple segmentation par des espaces et des signes de ponctuation conduit généralement à des résultats d'indexation médiocres.
- Par exemple, pour le français, nous avons :

2.1.1) La Segmentation

- Par exemple, pour le français, nous avons :
 - Les composés lexicaux **à trait d'union** comme *chassé-croisé, peut-être, rendez-vous*, etc.
 - Les composés lexicaux **à apostrophe** comme *jusqu'où, aujourd'hui, prud'homme*, etc.
 - Les **expressions idiomatiques** comme *au fait, poser un lapin, tomber dans les pommes*, etc.
 - Les **formes contractées** comme *Gad'zarts (les gars des Arts et Métiers), M'sieur, j'*, etc.
 - Les **sigles** et les **acronymes** comme *K7, A.R., CV, càd, P.-V.*, etc.

2.1.1) La Segmentation

- Dans ce cas, lorsque nous indexons un texte en français, nous ne souhaitons pas, par exemple, que le mot *aujourd'hui* soit séparé en *aujourd* et *hui* alors que nous voulons que les mots *un homme* et *l'homme* soient indexés sous le même terme, *homme*.

2.1.1) La Segmentation

- Ce problème devient même extrême avec l'allemand, où les noms composés s'écrivent sans espace ; par exemple le mot

Rindfleischetikettierungsüberwachungsaufgabenübertragungsgesetz

- Il doit, *à priori*, être séparé en les dix mots qui le constituent - Rind (*boeuf*) + Fleisch (*viande*) + Etikettierung (*étiquetage*) + Überwachung (*surveillance*) + Aufgabe (*tâche*) + Übertragung (*transmission*) + Gesetz (*droit*) - avant d'être indexé.

2.1.1) La Segmentation

- Pour les **langues européennes**, différents logiciels de segmentation commerciaux existent.
- Le but de certains de ces logiciels est de réaliser une analyse plus poussée du texte, en associant aux mots d'une phrase leur fonction grammaticale.
- La segmentation dans ces cas est une étape préliminaire à cette analyse.
- On peut citer ici le logiciel **TreeTagger** (Schmid 1994) qui est en libre accès et est couramment utilisé pour cette tâche.

2.1.1) *La Segmentation*

- *Exercice:*

Donnez-deux exemples de spécificités de segmentation rencontré dans votre langue africaine locale

2.1.2) La Normalisation

2.1.2) La Normalisation

- Introduction
- Un des obstacles majeurs à l'amélioration des performances des moteurs de recherche a été depuis longtemps le non appariement des termes de l'index et ceux des requêtes.
- Dans la plupart des cas, le problème vient d'un désaccord de forme lexicale entre les termes.

-

2.1.2) La Normalisation

- Introduction
- Par exemple, si le terme *P.-V.* figure dans l'index, nous souhaitons que, pour la requête constituée du mot *PV*, le système retourne aussi les documents répertoriés dans la liste d'index de *P.-V.*
- .

2.1.2) La Normalisation

- Introduction
- Définition : La **normalisation** de mots est le **processus** qui transforme tous les mots d'une même famille sous une forme normale ou canonique de façon à ce que l'appariement entre les termes d'une requête et ceux de l'index puissent avoir lieu, et ce malgré les différences qui peuvent exister entre les séquences de caractères de leurs mots associés.
- .

2.1.2) La Normalisation

- **Introduction**
- **Par exemple**, après normalisation les mots *P.V.*, *P.-V.* et *PV* seront mis sous une même forme normale et les requêtes formées par un de ces trois mots donneront alors le même résultat.
- Il existe deux types de normalisation :
 - la **normalisation textuelle**, ou *surfacique*, et
 - la **normalisation linguistique**.
- Chaque type donne lieu à des règles de normalisation différentes que nous présentons dans ce qui suit:

2.1.2) *La Normalisation*

- *Normalisation textuelle*
- La normalisation textuelle rend les mots d'une même famille sous leur forme canonique en effectuant quelques transformations superficielles sur les séquences de caractères de ces mots. L'exemple précédent avec *P.V.* constitue un cas de normalisation textuelle.

2.1.2) *La Normalisation*

- *Normalisation textuelle*
- Comme expliqué précédemment, la normalisation textuelle est dépendante de la langue.
- Les règles communément utilisées en français pour ce type de normalisation sont décrites ci-dessous.

2.1.2) La Normalisation

- *Les ponctuations*
- La règle de base appliquée à l'exemple ci-dessus, et qui concerne tous les acronymes, est d'enlever les points et les traits d'union apparaissant dans les mots.

2.1.2) *La Normalisation*

- *La casse*
- Une stratégie classique est de réduire certaines lettres en minuscules.
- Elle ne peut pas être généralisée à cause de certains noms propres (Blanc, Noir) et des acronymes (ACE - Association des cinémathèques européennes, CAS - Centre d'analyse stratégique) dont la réduction des majuscules en minuscules peut conduire à les confondre avec des noms communs qui s'écrivent pareillement (CAS ! cas).
- L'heuristique la plus souvent utilisée dans ce cas consiste à convertir les premières lettres des mots se trouvant au début de chaque phrase.

2.1.2) La Normalisation

- Les accents
- L'impact des accents sur le résultat de la recherche est souvent marginal.
- La règle appliquée consiste généralement à **enlever** les diacritiques sur tous les mots (par exemple *ambiguë* devient *ambigue* ou *forêt* qui devient *foret*).

2.1.2) La Normalisation

- **Les dates et les valeurs monétaires**
- Les règles pour normaliser ces types de mots sont assez spéciales et dépendent de la langue et des usages particuliers de chaque pays.
- Pour certaines langues comme l'anglais d'Amérique du Nord, le problème des dates est même plus compliqué puisque dans les documents d'une collection nous pouvons rencontrer des occurrences de

année/mois/jour

ou

mois/jour/année

pour une même date.

2.1.2) La Normalisation

- **Normalisation linguistique**
- La normalisation linguistique consiste à ramener un mot fléchi sous sa forme canonique.
- Il existe deux types de normalisation linguistique :
 - la *racinisation* et
 - la *lemmatisation*.

2.1.2) La Normalisation

- **La racinisation**
- La **racinisation** se rapporte au procédé qui cherche à regrouper les différentes variantes morphologiques d'un mot autour d'une même racine (en anglais **stem**).
- Ce procédé repose sur un ensemble de règles pour supprimer les flexions et les suffixes des mots, et il est fortement dépendant de la langue utilisée.

2.1.2) La Normalisation

- **La racinisation**
- **Par exemple**, les mots français suivant ont la même racine morphologique (en gras) :
- *chanter, chante, chantes, chantent, chanté, chantée, chants*) **chant**
- La racinisation peut aussi ramener des mots à des formes qui ne sont pas de vrais mots :
- *cheval, chevaux, chevalier, chevalerie, chevaucher*) **cheva**

2.1.2) La Normalisation

- **La racinisation**
- Les premiers algorithmes de racinisation, appelés **stemmer**, ont été étudiés à partir de la fin des années 1960 et les plus connus sont ceux de (Lovins 1968), (Porter 1980) et (Paice 1990).
- Ces stemmers sont conçus pour **l'anglais** et ils procèdent en traitant les mots par des étapes prédéfinies.
- À une étape donnée, on détermine si la terminaison d'un mot se trouve dans une liste prédéfinie.
- Le cas échéant, le stemmer la supprime ou la transforme suivant la règle à appliquer.
- D'autres algorithmes de racinisation pour d'autres langues ont été conçus suivant le même principe. Pour le **français**, on peut par exemple citer :

2.1.2) La Normalisation

- La racinisation
- Pour le français, on peut par exemple citer :
 - Le stemmer de Carry (Paternostre *et al.* 2002), développé dans le cadre du projet *Generic Analyser and Listener for Indexed and Linguistics Entities of Information* (GALILEI).
 - Le stemmer Snowball, disponible sur Internet et dont les règles de racinisation sont expliquées sur la page de téléchargement.

2.1.2) *La Normalisation*

- *La racinisation*
- En RI, le but de la racinisation est de trouver les documents contenant toutes les formes fléchies des mots d'une requête.
- En contrepartie, **le risque** est qu'on peut aussi ramener des documents qui contiennent les mots ayant les mêmes racines que les mots de la requête mais qui sont **sémantiquement totalement différents** de ces derniers.

2.1.2) La Normalisation

- La racinisation
- Par exemple,
 - Si la requête porte sur les **chevaux**, en considérant les documents contenant le mot normalisé **cheva**, on sait qu'on aura une bonne couverture du sujet, puisqu'on obtiendra les documents contenant toutes les occurrences des mots *cheval* et *chevaux*.
 - **Par contre**, dans la liste des documents retournés, figureront également ceux qui contiennent les mots *chevaucher*, *chevalerie* et *chevaliers* qui pourront être un peu **éloignés du sujet recherché**.

2.1.2) La Normalisation

- **La lemmatisation**
- La lemmatisation
 - fait une analyse linguistique poussée
 - destinée à enlever les variantes flexionnelles des mots afin de les
 - ramener sous leur forme lemmatisée ou encyclopédique.

2.1.2) La Normalisation

- La lemmatisation
- Cette transformation fournit
 - Pour les verbes, la forme à **l'infinitif**,
 - Pour les adjectifs la forme **masculin singulier** et
 - Pour les noms la forme singulier.

2.1.2) La Normalisation

- La lemmatisation
- À l'opposé de la racinisation,
 - le résultat de la lemmatisation n'est autre que des mots de l'encyclopédie ;
 - elle ne conduit ainsi pas à l'agrégation de mots très différents.
- Avec l'exemple de la requête précédente, **chevaux** sera agrégé à **cheval** mais pas à *chevalerie* ou *chevaucher*.

2.1.2) La Normalisation

- La lemmatisation
- Dans ce cas, si on lemmatise les mots, le résultat de la recherche fournira des documents contenant aussi bien les mots *cheval* que *chevaux* mais pas les documents qui comportent *chevalerie* ou *chevaucher*.

2.1.2) La Normalisation

- La lemmatisation
 - Pour le français et d'autres langues fortement fléchies comme l'allemand ou le finnois, la lemmatisation semble avoir un effet bénéfique,
 - mais ceci n'est pas toujours le cas avec des langues peu fléchies comme l'anglais et les procédures de racinisation, plus faciles à mettre en oeuvre, sont en général privilégiées.
- .

2.1.3) Filtrage par un antidictionnaire

2.1.3) Filtrage par un antidictionnaire

- **Définition** : Un **antidictionnaire** (*stopword list* en anglais), ou **liste de mots vides**, est une liste de mots qui tendent à être présents avec une fréquence élevée dans tous les documents d'une collection et qui n'apportent que peu d'information sur le contenu d'un document.

2.1.3) Filtrage par un antidictionnaire

- Le **filtrage** par un antidictionnaire est très pratiqué dans les tâches de **catégorisation** et de **partitionnement** de documents.
- En effet, la distribution des mots vides est **identique** dans tous les documents d'un corpus, ceci quelles que soient les thématiques auxquelles ils appartiennent, et les mots vides n'ont ainsi pas un pouvoir discriminant élevé pour ces tâches.
- L'idée de ce filtrage vient de (Luhn 1958), un des pionniers en RI.

2.1.3) Filtrage par un antidictionnaire

- Il existe des antidictionnaires
 - indépendants du domaine de la collection que l'on considère et
 - d'autres qui en sont dépendants.

2.1.3) Filtrage par un antidictionnaire

- **Les antidictionnaires dépendants d'un domaine sont construits**
 1. en triant d'abord les mots suivant leur nombre total d'apparitions dans la collection et
 2. en sélectionnant ensuite à la main les mots les plus fréquents et à faible contenu informatif.
- **Un exemple de mots vides** dépendants du domaine est le mot **projet** pour une collection de documents constituée de descriptifs de *projets*.

2.1.3) Filtrage par un antidictionnaire

- Pour les antidictionnaires généraux, c'est-à-dire **indépendants** des domaines considérés, on se repose soit sur la catégorie grammaticale des mots,
- l'antidictionnaire contiendra alors tous les mots des parties du discours **autres que nom, adjectif, verbe et adverbe**, soit sur un ensemble de collections de différents domaines d'une langue donnée, ensemble sur lequel on effectue le travail présenté au point précédent.

2.1.3) Filtrage par un antidictionnaire

de	la	le	et	en	l'	du
des	d'	les	est	un	une	il
au	dans	par	pour	sur	date	a
qui	que	avec	son	plus	se	sans
quel	quelle	s'	pas	n'	je	y
ou	se	sont	aux	qu'	sa	elle

Tab. 2,1 – Les 42 mots les plus fréquents (et peu informatifs)
présents dans la collection du Wikipédia français.

2.1.3) Filtrage par un antidictionnaire

- Une fois cette liste de mots constituée, on filtre les documents du corpus en enlevant les mots présents dans cet antidictionnaire.
- Le tableau 2.1 montre un exemple d'antidictionnaire relatif à la collection du Wikipédia français.

2.1.3) Filtrage par un antidictionnaire

- Après le filtrage des documents par un antidictionnaire, l'ensemble des *termes* restants constituent le vocabulaire de la collection
- C'est ce vocabulaire qui sera ensuite utilisé pour représenter les documents dans un espace vectoriel ou pour construire l'index inversé des termes.

2.2) Les deux lois de base en recherche d'information

2.2) Les deux lois de base en recherche d'information

- Introduction

1. On remarque d'abord la grande taille du vocabulaire de la collection.
 - Le Petit Robert contient 60 000 entrées et le Grand Robert 75 000,
 - le relevé le plus exhaustif montre que la langue française contiendrait à peu près 700 000 mots.
 - La question ici est alors de savoir pourquoi il existe un si grand nombre de mots différents (757 476) dans le Wikipédia français.

2.2) Les deux lois de base en recherche d'information

- Introduction

1. On remarque d'abord la **grande taille du vocabulaire** de la collection.
2. On note ensuite que :
 - le **filtrage** des documents par l'antidictionnaire, constitué des 200 mots vides, a réduit le nombre moyen d'occurrence de mots dans les documents de 416 à 225.
 - Autrement dit, près de **45%** des occurrences de mots dans les textes de Wikipédia correspond aux 200 mots les plus fréquents de la collection.
 - On constate également que leur **filtrage allège** l'espace disque de près de 39% (de 4,6 Go à 2,8 Go).

2.2) Les deux lois de base en recherche d'information

- Introduction
- Ces deux phénomènes sont expliqués par les **lois de Heaps** et de **Zipf** présentées dans ce qui suit.

2.2.1) Loi de Heaps

2.2.1) Loi de Heaps

- La loi de (Heaps 1978) stipule que la taille du vocabulaire (V) croît exponentiellement en fonction du nombre de mots présents dans une collection donnée (M). Autrement dit :

$$V = K \times M^{\beta} \quad (1)$$

- où K et β sont des paramètres dépendant de la collection considérée.

2.2.1) Loi de Heaps

- La loi de (Heaps 1978)
- La figure 2.1 montre l'évolution exponentielle de la taille du vocabulaire en fonction du nombre de mots de la collection de Wikipédia.
- En utilisant une régression des moindres carrés, on a trouvé que les paramètres de la loi sont, dans ce cas, approximativement égaux à $K \approx 95$ et $\beta \approx 0,43$.

2.2.1) Loi de Heaps

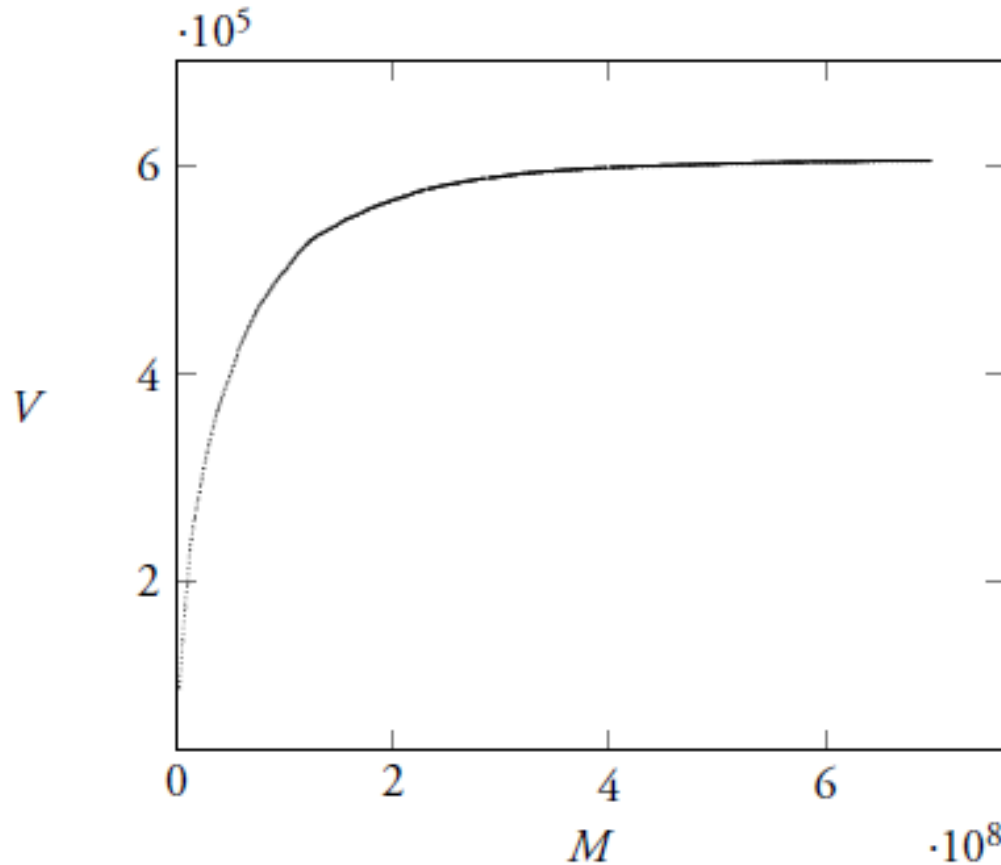


Fig. 2 ,1 : Illustration de la loi de Heaps sur la collection du Wikipédia français.

Dr Kiswendsida Kisito Kaboré.

<http://Kisitokab.xyz>

2.2.1) Loi de Heaps

- Le paramètre β dépend de la langue de la collection mais
- le paramètre K dépend beaucoup des prétraitements appliqués à une collection pour extraire les mots et les termes.
- En effet,
 - une mauvaise segmentation a un incidence directe sur les mots que l'on extrait (et donc sur leur nombre) et
 - des stratégies de normalisation vont bien réduire le nombre de types de mots (d'à peu près 21 % dans notre cas).

2.2.1) *Loi de Heaps*

- Quelles que soient les valeurs de ces paramètres, la loi de Heaps suggère que la taille du vocabulaire croît au fur et à mesure que la collection croît.

2.2.2) Loi de Zipf

2.2.2) Loi de Zipf

- 2.2.2 Loi de Zipf
- La loi de (Zipf 1935) spécifie que la **fréquence d'occurrence $fc(m)$ d'un mot m dans une collection de documents est inversement proportionnelle à son rang**, le rang étant ici celui obtenu lorsque l'on trie par ordre décroissant des fréquences les mots de la collection :

$$\forall m, \quad fc(m) = \frac{\lambda}{rang(m)} \quad (2)$$

- où λ est ici aussi un paramètre dépendant de la collection considérée.
-

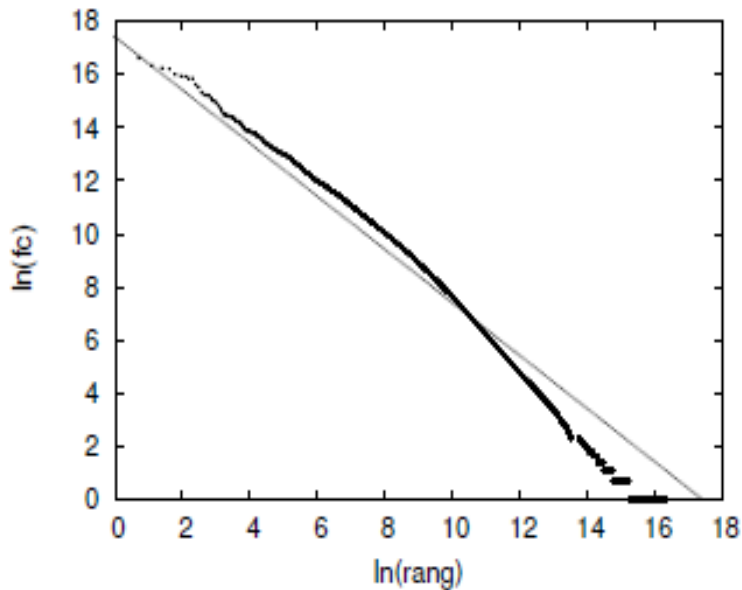
2.2.2) Loi de Zipf

- **2.2.2 Loi de Zipf**
- La figure 2.2 montre le lien entre les rangs et les fréquences d'occurrence des mots ainsi que les fréquences des mots de la collection de Wikipédia dont le rang vaut au plus dix.
- Sur la double échelle logarithmique nous voyons effectivement que la droite

$$\ln(fc) = 17,42 - \ln(\text{rang})$$

représente une bonne approximation de la relation entre logarithmes des fréquences et des rang des mots de cette collection.

2.2.2) Loi de Zipf



rang	mot	fréquence
1	<i>de</i>	36 875 868
2	<i>la</i>	16 565 726
3	<i>le</i>	12 639 034
4	<i>et</i>	11 587 487
5	<i>en</i>	10 885 221
6	<i>l'</i>	8 937 203
7	<i>du</i>	8 541 846
8	<i>des</i>	8 302 026
9	<i>d'</i>	7 766 576
10	<i>les</i>	7 722 951

Fig. 2.2 – Illustration de la loi de Zipf sur la collection du Wikipédia français (gauche).

2.2.2) Loi de Zipf

- **2.2.2 Loi de Zipf**
- **Une conséquence** de la loi de Zipf est que la fréquence du second mot le plus fréquent est la moitié de la fréquence du premier, etc.
- **Une autre conséquence** est que la fréquence des mots décroît rapidement avec leur rang.
- Ceci explique pourquoi le **filtrage** par les 200 mots les plus fréquents diminue si drastiquement la taille des documents de la collection de Wikipédia.

2.3) Représentation documentaire

2.3) Représentation documentaire

- **Introduction**
- En recherche documentaire, le but est de trouver les documents qui contiennent les termes d'une requête donnée.
- Pour d'autres tâches comme la classification ou le partitionnement, on cherche à discriminer ou à regrouper les documents entre eux.
- La représentation du contenu d'une collection est donc une phase primordiale dans tous les processus de l'accès à l'information et a motivé de nombreux travaux.
- Dans la suite nous allons présenter les techniques et algorithmes classiques pour la représentation documentaire avant de présenter les algorithmes classiques d'indexation.

2.3.1) Modèle vectoriel et représentation en sac de mots

2.3.1) *Modèle vectoriel et représentation en sac de mots*

- **Le modèle vectoriel** (ou *Vector Space Model*), proposé par Gerard Salton (Salton 1975), définit la représentation documentaire communément utilisée dans différentes tâches de l'accès à l'information.
- Avec cette représentation on associe à chaque document d d'une collection C , un vecteur $\mathbf{d}$, dont la dimension correspond à la taille du vocabulaire.
- L'espace vectoriel considéré est donc un espace vectoriel de termes dans lequel chaque dimension est associée à un terme de la collection.

$$\forall d \in C, \quad \mathbf{d} = (w_{id})_{i \in \{1, \dots, V\}} \quad (3)$$

2.3.1) *Modèle vectoriel et représentation en sac de mots*

- Le modèle vectoriel

$$\forall d \in \mathcal{C}, \quad d = (w_{id})_{i \in \{1, \dots, V\}} \quad (3)$$

- Dans ce cas, w_{id} est le poids que le terme d'indice i du vocabulaire a dans le document d .
- Nous verrons plus loin qu'il existe plusieurs façons de calculer ce poids.
- Il est toutefois important de noter ici que toutes ces méthodes assignent un poids nul à un terme non présent dans le document.
- Avec cette représentation, l'ordre d'apparition des termes dans les documents n'est pas pris en compte (figure 2.3.1).
- Pour cette raison, cette représentation est aussi connue sous le nom de la *représentation par sac de mots*.
- Au niveau terminologique, la matrice, X_{VN} composée des vecteurs des documents représentés dans l'espace des termes du vocabulaire est notée la matrice termes-documents.

2.3.1) Modèle vectoriel et représentation en sac de mots

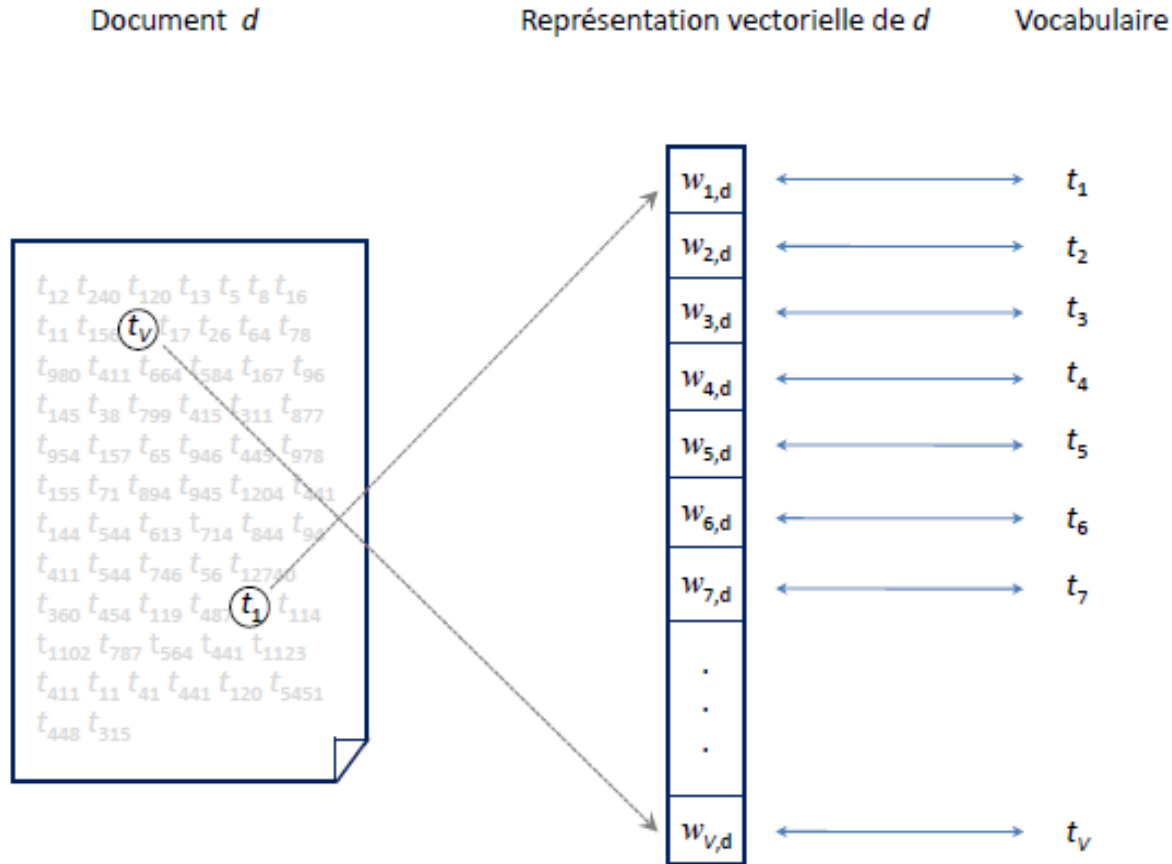


Fig. 2.3.1 – Représentation par sac de mots. Dans le document d , les indices des termes correspondent à leur indices dans la liste des termes constituant le vocabulaire.

2.3.1) *Modèle vectoriel et représentation en sac de mots*

- La représentation vectorielle (et les poids qu'elle manipule en général) implique que :
 - les documents contenant des termes sémantiquement proches mais n'utilisant pas les mêmes termes qui se trouvent sur des dimensions différentes de l'espace vectoriel,
 - les vecteurs de documents contiennent peu de valeurs non nulles (*exemple : 0,05% de caractéristiques non nulles en moyenne pour les documents de la collection de Wiképdia*)
- Pour ne pas stocker inutilement les valeurs nulles présentes dans les vecteurs de représentation des documents, ces derniers sont souvent codés en ne gardant que les indices correspondant aux valeurs non nulles.

2.3.2) Pondération des termes

2.3.2) Pondération des termes

- **2.3.2 Pondération des termes**
- Malgré ces limitations, cette représentation en sac de mots est largement utilisée en catégorisation et partitionnement de documents ainsi que dans des modèles vectoriels de recherche.
- Elle est de plus très similaire à celle utilisée dans les modèles probabilistes de recherche d'information et présente l'avantage d'être facilement manipulable et exploitable par des techniques statistiques génériques.

2.3.2) Pondération des termes

- **2.3.2 Pondération des termes**
- Une approche simple et classique pour estimer **les poids (w_{id})** du vecteur associé au document d est de calculer le nombre d'occurrences des termes du vocabulaire dans le document.
- Cette approche connue en anglais sous le nom de **term frequency (tf)** définit ainsi chaque poids associé à un terme par le nombre de fois où ce terme apparaît dans un document :

$$\forall i \in \{1, \dots, V\}; \quad w_{id} = tf_{t_i, d}$$

2.3.2) Pondération des termes

- **2.3.2 Pondération des termes**
- Cela dit, cette représentation fréquentielle introduit un biais problématique, les documents longs tendant à comporter davantage de répétitions de termes que les courts.
- Ainsi, un mot qui apparaît 10 fois dans un document court (de 100 mots par exemple) est bien plus important pour ce document que le même mot apparaissant 100 fois dans un document très long (de 100 000 mots par exemple), différence dont ne rend pas compte le poids précédent (qui conduit en fait à l'importance inverse). Comme nous le verrons plus loin, différents types de normalisation permettent de pallier ce problème.

2.3.2) Pondération des termes

- Ceci étant, avec cette approche, tous les termes du vocabulaire apparaissant dans un document avec la même fréquence recevront le même poids.
- Or, les termes qui figurent dans peu de documents permettent de mieux différencier ces derniers des documents dans lesquels ils se trouvent en nombre élevé.
- En d'autres mots, ces termes ont un pouvoir de discrimination plus grand que celui des termes qui apparaissent dans beaucoup de documents.
- **Par exemple**, dans la collection du Wikipédia français, le terme *algèbre* est spécifique à un domaine particulier, et sa présence dans un document donne une meilleure indication quant au sujet qui y est traité que le terme *calcul*, qui est un terme commun à plusieurs domaines présents dans la collection.

-

2.3.2) Pondération des termes

- Dans le calcul des poids, on prend en compte le nombre de documents dans lesquels un terme t du vocabulaire apparaît, appelé *document frequency* en anglais et usuellement noté par df_t , en multipliant le nombre d'apparitions du terme t dans un document d donné par son idf_t , le logarithme de l'inverse normalisé de df_t :

$$idf_t = \ln \frac{N}{df_t}$$

- où N est le nombre de documents de la collection.

2.3.2) Pondération des termes

- En recherche d'information, le codage le plus courant des documents, connu sous le nom du **codage *tf-idf*** est défini comme :

$$\forall i \in \{1, \dots, V\}; \quad w_{id} = tf_{t_i,d} \times idf_{t_i} \quad (4)$$

- De très nombreuses variantes de ce codage ont été proposées et elles ont fait l'objet d'un grand nombre de comparaisons expérimentales.
- Elles utilisent souvent un terme de normalisation, n ., dépendant de la taille du document considéré, en plus des deux termes à base de tf et de idf .

2.3.2) Pondération des termes

- La forme générale du codage des termes est ainsi :

$$\forall i \in \{1, \dots, V\}; \quad w_{id} = n_d \text{Ptf}_{t_id} \times \text{Pdf}_{t_i} \quad (4)$$

- Les w_{id} sont des poids fondés sur la fréquence du terme t_i dans d , (tfti, d), ainsi que sur le nombre de documents le contenant (dfti).
- Le tableau 2.3 récapitule les variantes les plus communes, proposées dans SMART le système de RI développé par le groupe de Gerard Salton (Salton 1975), et nous allons présenter dans la suite les trois pondérations à base de fréquence des termes les plus utilisées dans la littérature.

2.3.2) Pondération des termes

$$\forall d \in C, \forall i \in \{1, \dots, V\}, w_{id} = n_d \times p_{tf_{i,d}} \times p_{df_{i_i}}$$

$p_{tf_{i_i,d}}$	$p_{df_{i_i}}$	n_d
$tf_{i_i,d}$	1	1
$\frac{1 + \ln(tf_{i_i,d})}{1 + \ln(moy_tf(d))}$	idf_{i_i}	$\frac{1}{\ d\ } = \frac{1}{\sqrt{\sum_{i=1}^V w_{id}}}$
$\begin{cases} 1 + \ln(tf_{i_i,d}) & \text{si } tf_{i_i,d} > 0, \\ 0 & \text{sinon.} \end{cases}$		
$0,5 + 0,5 \times \frac{tf_{i_i,d}}{tf_{\max_d}}$	$\max\{0, \ln \frac{N - df_{i_i}}{df_{i_i}}\}$	$\frac{1}{(Char_d)^\alpha}, 0 < \alpha < 1$
$\begin{cases} 1 & \text{si } tf_{i_i,d} > 0, \\ 0 & \text{sinon.} \end{cases}$		

Tab. 2.3 – Différentes variantes du codage tf-idf, proposées dans SMART (Salton 1975).

Char_d correspond à la taille du document d, en nombre de caractères le constituant.

2.3.2) Pondération des termes

- **A-La Pondération fréquentielle normalisée**
- Comme nous l'avons noté plus haut, la pondération fréquentielle est très sensible aux variations de la taille des documents.
- Par exemple, si on triple la taille d'un document, les poids des termes seront triplés et ce document augmenté sera considéré comme plus pertinent pour une requête composée de certains de ses termes que le document de départ.

-

2.3.2) Pondération des termes

- **A-La Pondération fréquentielle normalisée**
- Une façon générale de remédier à ce problème est de normaliser les fréquences des termes apparaissant dans un document d par la fréquence maximale dans le document $tf_{max_d} = \max_{t \in d} tf_{t,d}$.
- $\forall i \in \{1, \dots, V\}; P_{tf_{t_i,d}} = \lambda + (1 - \lambda) \times \frac{tf_{t_i,d}}{tf_{max_d}}$
- où $\lambda \in [0, 1[$ est un terme de lissage, généralement fixé à 0,5.

2.3.2) Pondération des termes

- **B- La Pondération logarithmique des termes**
- Dans le cas de documents de grande taille, la pondération *tf-idf* linéaire privilégie le terme *tf* lorsque ce dernier est grand, alors même que la répétition d'un terme n'est pas le seul facteur déterminant son importance dans le calcul du poids, comme nous l'avons vu plus haut.
- Pour diminuer cet effet, et ainsi la disparité entre les caractéristiques du vecteur de représentation associé à un document *d*, on utilise souvent le logarithme du nombre d'occurrences des termes (Buckley *et al.* 1992) :

$$\bullet \forall i \in \{1, \dots, V\}; P_{tf_{t_i,d}} = \begin{cases} 1 + \ln(tf_{t_i,d}) & \text{si } tf_{t_i,d} > 0 \\ 0 & \text{sinon} \end{cases}$$

2.3.2) Pondération des termes

- **C- La Pondération logarithmique normalisée**
- Pour tenir compte des documents de petite taille, plusieurs études ont proposé de normaliser la caractéristique logarithmique précédente par une quantité qui dépend de la moyenne des caractéristiques fréquentielles d'un document d donné, $\sum_{i=1}^X \frac{tf_{t_i,d}}{|d|}$, où $|d|$ représente le nombre de termes du vocabulaire différents dans le document d (si d ne contient qu'un seul terme, qui apparaît 3 fois, $|d| = 1$) :
- $\forall i \in \{1, \dots, V\}; P_{tf_{t_i,d}} = \frac{1 + \ln(tf_{t_i,d})}{1 + \ln(moy_tf(d))}$
- Après avoir exposé la représentation classique des documents sous forme vectorielle, nous allons maintenant nous intéresser à la construction de l'index inversé, qui fournit, pour chaque terme, la liste des documents qui le contiennent.

2.4) Index inversé

2.4) Index inversé

- **Introduction**
- L'utilisation **d'une structure** qui fait correspondre chaque terme du vocabulaire à la liste des documents qui le contiennent est la façon la plus rapide pour trouver un terme d'une requête donnée dans une collection de documents.
- **Cette structure s'appelle communément un index inversé** et elle peut prendre plusieurs formes.
- **La forme la plus simple** est celle dans laquelle, pour chaque terme du vocabulaire, *l'indexeur* ne fournit que la liste des identifiants de documents où ce terme apparaît.
- D'autres index indiquent aussi le **nombre de documents** contenant les termes, **df**, leur **nombre d'occurrences** dans les documents, **tf**, ou la position des termes dans les documents.

2.4) Index inversé

- **Introduction**
- Les **deux types de structure de données** les plus utilisées pour la construction de l'index inversé sont des **tables de hachage** ou des **arbres**.
- Pour ces deux types de données, la reconstruction de l'index en cas de modification de la collection est possible et leurs complexités sont généralement d'ordre linéaire sur le nombre de documents modifiés.
- Avec les arbres, il est toutefois possible de mener des recherches sur des termes tronqués (par exemple par leur préfixe) alors que les tables de hachage ne donnent pas cette possibilité.
- En revanche, la consultation sur les tables de hachage est beaucoup plus rapide.

2.4) *Index inversé*

- **Introduction**
- Pour un meilleur rendement, les systèmes de recherche mettent généralement le vocabulaire et une partie des listes des identifiants de documents dans la mémoire vive, les autres informations étant stockées sur disque.
- Avec l'augmentation continue des capacités de mémoire, il est possible de charger de plus en plus d'informations dans la mémoire, mais pour des très grandes collections le stockage du vocabulaire et des listes de documents dans la RAM nécessite de passer par des techniques de compression de données.

2.4) Index inversé

- Introduction
- Les premiers algorithmes d'indexation ont été proposés pour des **collections statiques** où les documents ne sont pas modifiés, supprimés, ajoutés au cours du temps.
- Avec la production continue de documents électroniques, nous sommes maintenant dans des situations où les documents d'une collection sont sujets à des modifications et suppressions quotidiennes (cas **dynamique**).
- Dans la suite nous allons présenter les algorithmes types pour chacun de ces deux cas, en distinguant les deux algorithmes d'indexation les plus utilisés dans les collections statiques à savoir ***l'indexation par bloc à base de tri*** et ***l'algorithme MapReduce*** pour le cas distribué.

2.4.1) Indexation dans des collections statiques

- **2.4.1 Indexation dans des collections statiques**
- Les différentes étapes de création d'index inversé des termes pour une collection statique de documents sont :
 1. L'extraction ,
 2. Le tri,
 3. Le regroupement.

2.4.1) Indexation dans des collections statiques

- **2.4.1 Indexation dans des collections statiques**
- **1. L'extraction** de paires d'identifiants (terme, document), en opérant une passe complète sur la collection. Ces identifiants sont généralement des clés numériques uniques associées à chaque terme et chaque document de la collection.
- Dans la figure 2.5, on a assimilé les identifiants des documents à leur indices et aussi chaque terme à son identifiant pour plus de clarté.
- 2. Le **tri**.
- 3. Le **regroupement**.

2.4.1) Indexation dans des collections statiques

- **2.4.1 Indexation dans des collections statiques**
- 1. L'**extraction**.
- **2. Le tri** des paires suivant les clés d'identifiants de termes puis les clés d'identifiants de documents.
- 3. Le **regroupement**.

2.4.1) Indexation dans des collections statiques

- 2.4.1 Indexation dans des collections statiques
- 1. L'extraction.
- 2. Le tri
- 3. Le regroupement des paires en construisant pour chaque identifiant de terme, la liste des identifiants de documents dans lesquels le terme apparaît. À ce stade, nous pouvons ajouter dans cette liste d'identifiants de documents d'autres informations comme le df ou le tf des termes.

2.4.1) Indexation dans des collections statiques

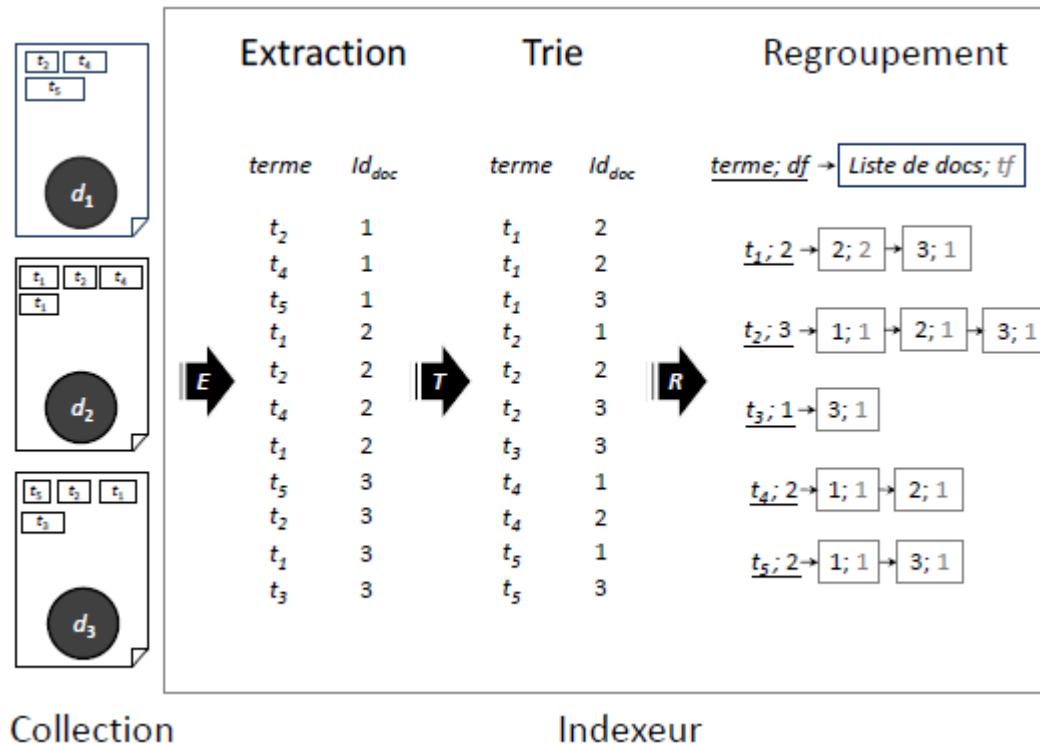


Fig. 5 – Création de l'index inversé pour une collection statique constituée de 3 documents, $C = \{d_1, d_2, d_3\}$ et de 5 termes, $V = \{t_1, t_2, t_3, t_4, t_5\}$. On suppose ici que l'ordre alphabétique est induit par l'ordre entre les indices des termes, i.e. le terme t_1 est avant le terme t_2 , etc

2.4.1) Indexation dans des collections statiques

- Indexation par bloc à base de tri
- L'étape la plus complexe suivant ce schéma est la phase de tri qui pour des grandes collections de documents ne peut pas être réalisée en mémoire.
- Dans ce cas, pour éviter de stocker la liste complète des paires à trier sur le disque et d'avoir recours à des accès disque gourmands en temps:
 - on stocke les paires d'identifiants (terme, document) par bloc en mémoire et
 - on effectue le tri avant de passer au bloc suivant.

2.4.1) Indexation dans des collections statiques

- Indexation par bloc à base de tri
- Le pseudo-code de cet algorithme, connu sous le nom **d'indexation par bloc à base de tri** ou ***blocked sort-based indexing*** en anglais, est donné dans l'algorithme 1.
- Cet algorithme parcourt la collection de documents et collecte les paires d'identifiants de termes et de documents dans un tampon (*buffer* en anglais) de mémoire de taille fixe.

2.4.1) Indexation dans des collections statiques

- **Indexation par bloc à base de tri**
- Si on a parcouru toute la collection ou que le buffer est plein, on arrête le parcours et on trie la liste des paires contenue dans le buffer, d'abord suivant la clé sur les identifiants de termes puis suivant la clé sur les identifiants de documents.
- On regroupe ensuite les paires ainsi triées pour constituer la liste des identifiants de documents par terme et on stocke ces listes sur le disque.
- Ce procédé est répété, jusqu'à ce que tous les documents de la collection aient été parcourus.
- À la fin du parcours, l'index inversé est obtenu en fusionnant toutes les listes d'identifiants de documents stockées sur le disque.

2.4.1) Indexation dans des collections statiques

- **Indexation par bloc à base de tri**
- La fusion finale est conduite en ouvrant simultanément toutes les listes d'identifiants de documents intermédiaires et en gardant un pointeur sur les fichiers associés à chacune de ces listes.
- À une étape donnée, on choisit l'identifiant de terme le plus petit qui n'a pas été encore traité.
- On fusionne alors toutes les listes correspondant à cet identifiant de termes dans tous les fichiers ouverts et l'on écrit sur disque la liste fusionnée.

2.4.1) Indexation dans des collections statiques

- Indexation par bloc à base de tri
- Cette fusion implique une lecture des listes d'identifiants de documents pour le terme traité, et donc un déplacement du pointeur de fichier correspondant.
- Si un terme n'est pas présent dans un fichier, le pointeur de ce dernier n'est pas modifié.
- Dans la mesure où les éléments dans les différents fichiers sont d'abord triés par identifiant de terme croissant, puis par identifiant de document croissant, l'étape précédente peut être répétée jusqu'à ce que tous les éléments aient été lus.
- Le temps d'indexation est dominé ici par le temps qu'il faut pour parcourir tous les documents ainsi que le temps nécessaire à la fusion de la liste intermédiaire des identifiants de documents.

2.4.1) Indexation dans des collections statiques

ALGORITHME 1. Algorithme d'indexation par bloc à base de tri

Entrée : une collection de documents prétraitée C

Initialisation : $b \leftarrow 0$; $pos \leftarrow$ debut de C

répéter

$b \leftarrow b + 1$;

$buffer \leftarrow \emptyset$;

répéter

$buffer \leftarrow buffer \cup \{\text{paires d'identifiant}(IDterm_{pos}, IDdoc_{pos})\}$;

$pos \leftarrow next_{pos}$;

jusqu'à $(buffer = \text{plein}) \vee (pos = \text{fin de } C)$;

$L_b \leftarrow \text{Tri}(buffer)$; // Les paires sont triées d'abord suivant les clés

 // d'identifiants de termes puis suivant les clés

 // d'identifiants de documents

$R_b \leftarrow \text{Regrouper}(L_b)$;

 Écrire R_b sur le disque ;

jusqu'à $pos = \text{fin de } C$;

Sortie : index inversé de C en fusionnant $R_1, \dots, R_b$

2.4.1) Indexation dans des collections statiques

- Indexation distribuée
- Pour des collections de documents très volumineuses comme la Toile (ou Web en anglais), l'indexation ne peut pas être effectuée sur une seule machine.
- Dans ces cas, on utilise un cluster de calcul et de traitement de données en mode distribué pour construire l'index inversé des termes.
- Au début des années 2000, **Google** a proposé un algorithme à base de **MapReduce** (Dean et Ghemawat 2008) pour réaliser ce type d'indexation.

2.4.1) Indexation dans des collections statiques

- Indexation distribuée
- Comme l'architecture **MapReduce**, cet algorithme s'articule en deux étapes.
 - **Dans la phase Map**, la collection de données est divisée en différentes parties dont chacune peut être traitée rapidement par un **ordinateur** (appelé **analyseur**).
 - **Dans l'étape Reduce**, chaque intervalle de paires d'identifiant de termes et de documents est ensuite assigné par un serveur à des **ordinateurs** qui vont trier les paires suivant la clé d'identifiant de terme et de document (appelés **inverseurs**)

2.4.2) Indexation dans des collections dynamiques

- 2.4.2 Indexation dans des collections dynamiques
- L'indexation dynamique consiste à actualiser l'index après **ajouts, modifications** ou **suppressions** des documents d'une collection dynamique.
- Une solution triviale à ce problème est de construire périodiquement un nouvel index et de transférer la recherche sur ce dernier une fois sa construction terminée.

2.4.2) Indexation dans des collections dynamiques

- 2.4.2 Indexation dans des collections dynamiques
- Le problème majeur avec cette stratégie est qu'elle est généralement très **coûteuse en terme de temps**, et pour des collections hautement dynamiques, comme la Toile, les performances d'un système de recherche à base de cette stratégie peuvent se dégrader rapidement pendant le laps de temps qui sépare la mise à jour de la collection et celle de l'index.
- Différentes stratégies d'indexation dynamique ont été proposées jusqu'à présent et la recherche suivant cet axe est encore très active.

2.4.2) Indexation dans des collections dynamiques

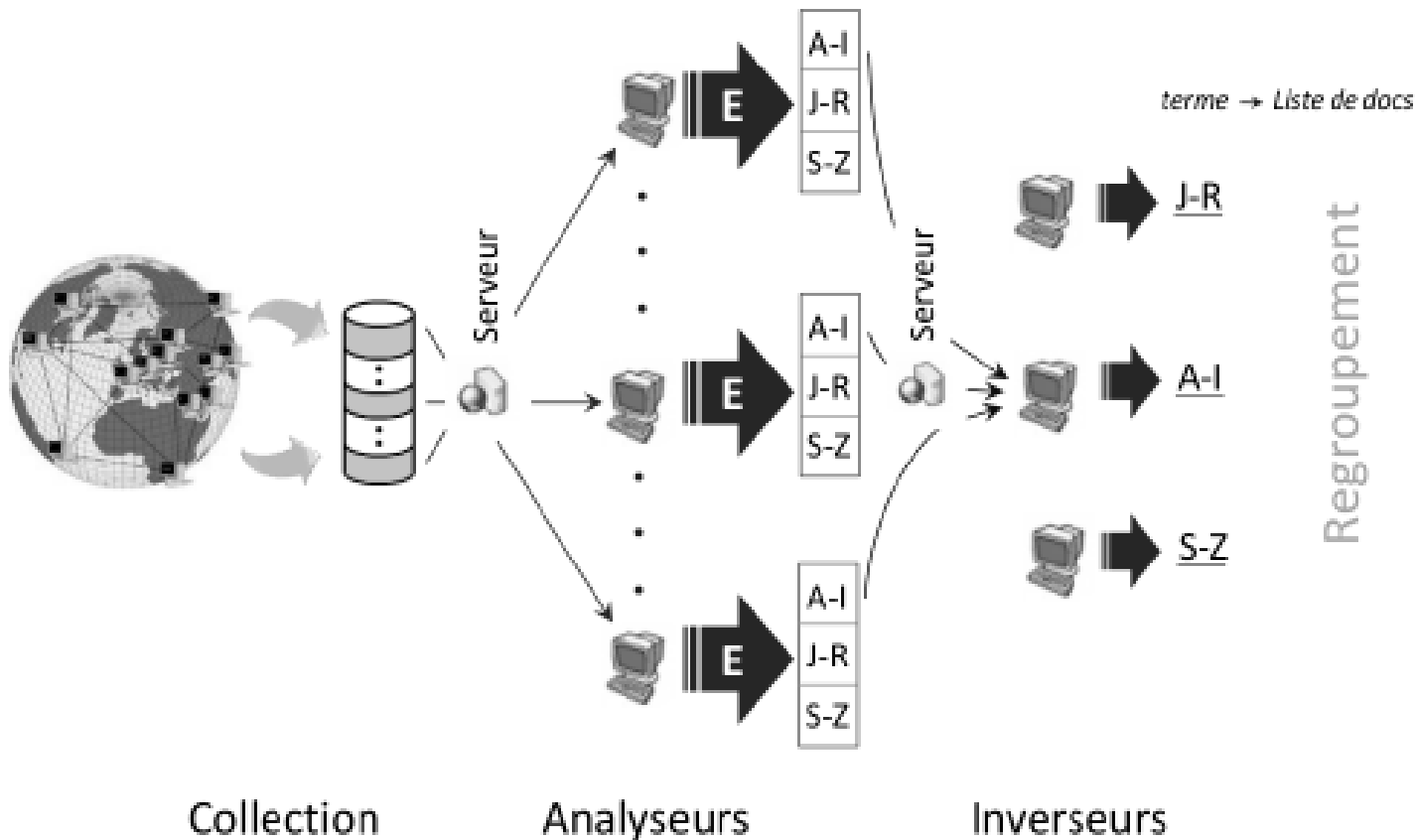


Fig. 6 – Illustration de l'indexation distribuée. Nous supposons ici que l'indexation de la collection a lieu lorsque cette dernière ne subit pas de modifications dans le temps.

2.4.2) Indexation dans des collections dynamiques

- 2.4.2 Indexation dans des collections dynamiques
- Les deux méthodes les plus utilisées sont :
 - la **technique de mise à jour directe sur place** (ou *In-Place strategy* en anglais) et
 - la **technique de fusion** (*Merge-based*).

2.4.2) Indexation dans des collections dynamiques

- La technique de mise à jour directe sur place
 - Cette technique ajoute directement les nouvelles listes d'identifiants de documents des termes aux listes d'identifiants de documents existantes dans l'index en cours.
 - Avant l'ajout, il faut s'assurer que l'espace disque disponible est suffisant pour contenir la nouvelle liste.
 - Dans le cas contraire, l'ancienne liste concaténée à la nouvelle doit être déplacée vers un nouvel emplacement où le tampon en mémoire contenant la nouvelle liste puisse être enregistré.
 - La complexité de cette stratégie réside donc dans sa gestion de l'espace libre sur le disque.

2.4.2) Indexation dans des collections dynamiques

- La technique de mise à jour directe sur place
- Un exemple de cette mise à jour est illustré dans la figure 7.
 - L'index en cours occupe un espace contigu sur le disque.
 - Les termes du vocabulaire pointent chacun sur leur listes d'identifiants de documents, les chiffres en tête de chaque liste et les boîtes $LD(t.)$ représentent respectivement les df et les listes d'identifiants de documents des termes.
 - Les boîtes marquées *libre* symbolisent l'espace libre dans une liste d'un terme donné.

La technique de mise à jour directe sur place

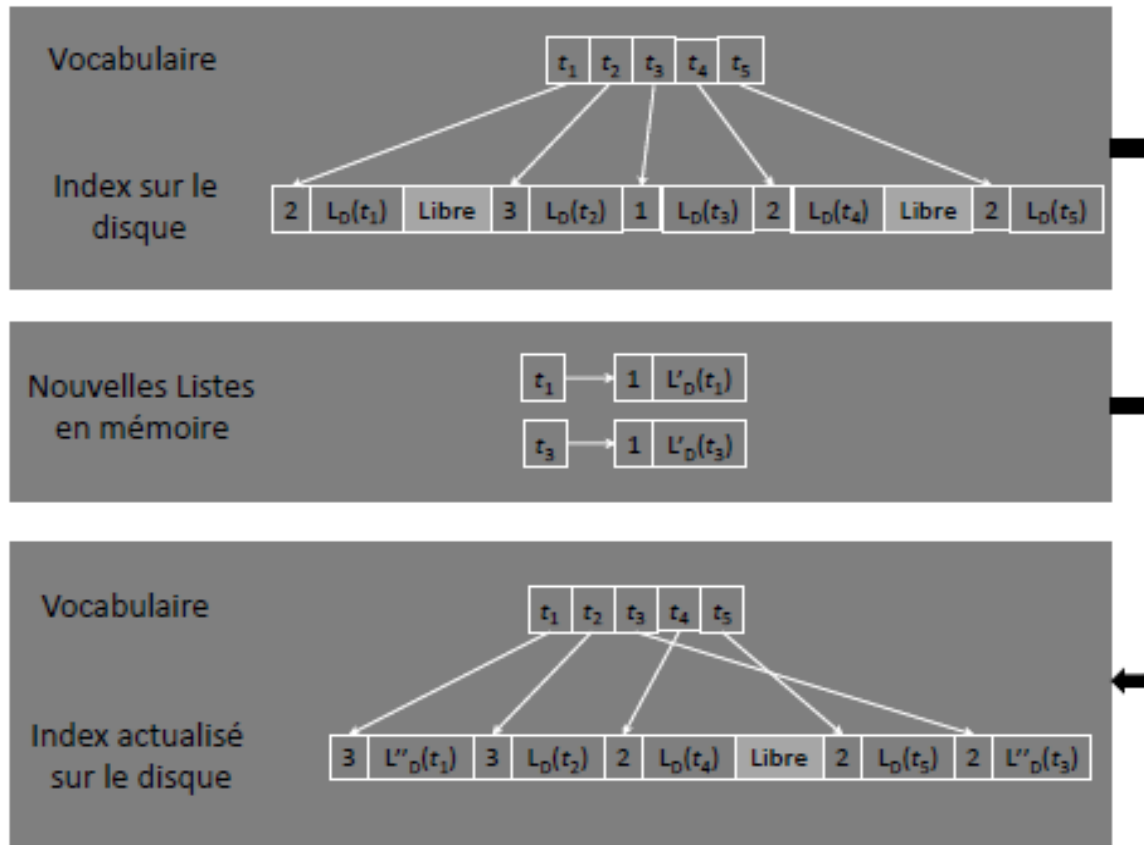


Fig. 7 –
Illustration
de la
stratégie
de mise à
jour directe
sur place.

2.4.2) Indexation dans des collections dynamiques

- La technique de mise à jour directe sur place
 - Lorsqu'un index temporaire est créé suite aux modifications de la collection, on regarde s'il reste de l'espace à la fin de la liste de documents d'un terme pour lequel une nouvelle liste est disponible en mémoire.
 - Si c'est le cas (ici pour le terme t_1 par exemple), on ajoute la nouvelle liste (notée par L' dans la figure) à la fin de l'ancienne sur le disque en changeant les statistiques concernées (ici le df).
 - Si ce n'est pas le cas la liste additionnée ainsi obtenue (notée par L'') est déplacée vers un nouvel emplacement (ici pour le terme t_3).

2.4.2) Indexation dans des collections dynamiques

- La technique de fusion

- La stratégie à base de fusion consiste à garder un index auxiliaire en mémoire à côté de l'index principal et de fusionner les deux, une fois que la taille de l'index auxiliaire devient trop grande.
- Après la fusion, un nouvel index sera entièrement créé sur le disque et l'ancien index ainsi que l'index auxiliaire en mémoire seront supprimés.
- Cette stratégie conserve un bit d'invalidation sur des documents supprimés et utilise cette information pour présenter ou non un document dans la liste retournée correspondant à une recherche.

2.4.2) Indexation dans des collections dynamiques

- La technique de fusion
- La figure 8 illustre cette technique.
- L'avantage de cette approche est que lors de la fusion, l'ancien index inversé peut être sollicité pour répondre à des requêtes.
- En contrepartie, l'inconvénient de la méthode est sa grande complexité dans la maintenance et la mise en œuvre, puisqu'à chaque mise à jour la totalité de l'index inversé sera lu et écrit sur le disque.

La technique de fusion

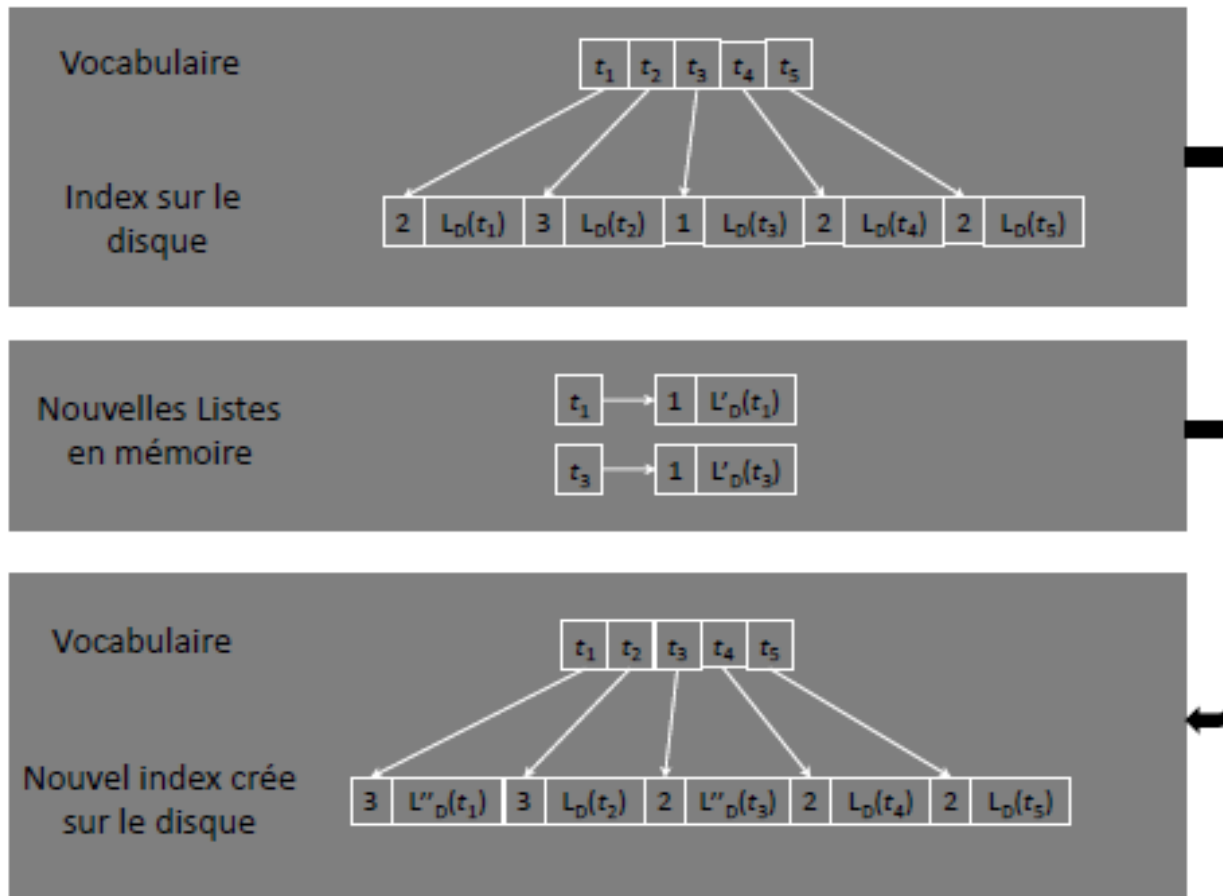


Fig. 8 – Illustration de la stratégie de fusion

<http://www.iro.umontreal.ca/~nie/IFT6255/>